MIDDLESEX UNIVERSITY
THE BURROUGHS, HENDON LONDON NW4 4BT

TITLE: CST2550 TECHNICAL REPORT

KAY BUGINGO: M00971312

NIGEL GAMBIZA: M00951628

CHOTA FIKANSA CHANDA: M01034342

ROHAN HAULKHORY: M01004090

KOSHAL MOHESS: M00943654

Table of Contents

1. Introduction 3
1.1 Project Overview 3
2. Design 3
2.1 System Architecture 3
2.2 Data Structure Selection and Justification 4
2.3 Algorithm Design (Pseudocode) 4
2.5 Database Design 5
3. Testing 6
3.1 Testing Approach 6
3.2 Test Cases Table 6
4. AI Agent 7
4.1 AI Agent Design 7
4.2 Implementation 7
5. Conclusion 8
5.1 Summary of Work Done 8
5.2 Limitations 8
5.3 Critical Reflection 8
6. References 8

1. Introduction

1.1 Project Overview

This report documents the design, development, and testing of a Takeaway Restaurant Management System, developed by a team of five students using Scrum methodology, with roles assigned as Scrum Master, Secretary, two Developers, and a Tester.

The system addresses the need to efficiently manage a restaurant menu and process orders in a fast-paced environment. The primary users are restaurant staff. Core functionality includes:

- Viewing, adding, removing, and searching menu items stored in a custom Binary Search Tree (BST).
- Placing customer orders by selecting items from the live menu.
- Tracking and updating order status (Pending, Preparing, Out for Delivery, Completed).
- Persisting all menu data to and from a SQLite database between sessions.
- Allowing the user to specify the database file path at runtime — no hardcoded file names.

The focus of the implementation is on the design and performance of the underlying data structures and algorithms, particularly the BST used to store and search menu items efficiently.

2. Design

2.1 System Architecture

The system is structured as an ASP.NET Core MVC web application (Microsoft, 2024a), organised around five main components. ASP.NET Core MVC was adopted to support the web-based chat interface required by the AI agent, role-based user sessions for Admin and Customer access, and multi-page navigation.

Data Classes

MenuItem stores an integer ID, name, category, price, and description. Order stores an order ID, customer name, phone number, address, an array of ordered items, total price, order date, and status.

Data Structures

Two custom BSTs store data in memory at runtime. *BinarySearchTree* holds *MenuItem* objects ordered by ID, supporting insert, delete, search by ID/name/category, in-order traversal, and count. *OrderBST* holds Order objects ordered by order ID, supporting insert, search, display, and count. Both use dedicated node classes (*BSTNode*, *OrderNode*) with left and right child pointers. Both BST instances are registered as singletons in *Program.cs*, shared across all web requests.

Database Layer

DatabaseHelper manages all SQLite communication. On startup, the database is initialised and all menu items are loaded into the BST. Changes are written back immediately on add, remove, or order placement.

Web Controllers

AccountController handles login/logout. *AdminController* manages menu items, orders, and status updates. *CustomerController* handles menu browsing, search, order placement, and order history. *AIController* receives POST requests from the chat page and returns AI-generated responses as JSON.

Data Flow

On startup: database is read, both BSTs are populated. During a session: controllers call BST operations in response to user actions. Orders and menu changes are persisted immediately. On chat requests: the AI agent queries the live BST directly.

2.2 Data Structure Selection and Justification

Linked List — Rejected

Search, positional insert, and delete all run at O(n), requiring full traversal. Unacceptable for a system requiring frequent menu lookups.

Hash Table — Rejected

Offers O(1) average search but has no inherent ordering. Displaying a sorted menu requires O(n log n) additional sorting. Hash collisions can also degrade performance to O(n) worst case.

Binary Search Tree — Selected

A BST organises data hierarchically: each node has at most two children, with smaller values left and larger right. This enables O(log n) average search, insert, and delete. Selected for the following reasons (Cormen et al., 2022):

- Search by ID runs at O(log n) average — significantly faster than O(n) for a frequently searched menu.
- In-order traversal displays all items in sorted order in O(n) — no additional sorting required.
- Insert and delete run at O(log n) average, efficiently supporting menu management.
- Category search requires O(n) traversal regardless of structure — the BST handles this cleanly via recursion.

A second BST (*OrderBST*) stores orders by ID, enabling O(log n) order retrieval and updates.

Time Complexity Summary

Operation	Linked List	Hash Table (avg)	BST (avg)	BST (worst)
Search by ID	O(n)	O(1)	O(log n)	O(n)
Insert	O(1)	O(1)	O(log n)	O(n)
Delete	O(n)	O(1)	O(log n)	O(n)
Ordered Display	O(n log n)	O(n log n)	O(n)	O(n)
Search by Name	O(n)	O(n)	O(n)	O(n)

Worst Case and Space Complexity

A standard BST degrades to O(n) if items are inserted in sorted order, producing an unbalanced tree. An AVL tree would prevent this, but given menu items are inserted in varied, non-sequential order and the dataset is small, this is unlikely. Space complexity is O(n) — one node per item — comparable to a Linked List and appropriate for this system's scale.

2.3 Algorithm Design (Pseudocode)

The following pseudocode describes the key BST algorithms in language-agnostic form.

INSERT — O(log n) avg, O(n) worst

```
INSERT(node, item):
  IF node IS NULL THEN RETURN new BSTNode(item)
  IF item.Id < node.Data.Id THEN node.Left = INSERT(node.Left, item)
  ELSE IF item.Id > node.Data.Id THEN node.Right = INSERT(node.Right, item)
```

```
  ELSE node.Data = item   // duplicate: update
  RETURN node
```

SEARCH BY ID — O(log n) avg, O(n) worst

```
SEARCH_BY_ID(node, id):
  IF node IS NULL THEN RETURN NULL
  IF node.Data.Id == id THEN RETURN node
  IF id < node.Data.Id THEN RETURN SEARCH_BY_ID(node.Left, id)
  ELSE RETURN SEARCH_BY_ID(node.Right, id)
```

SEARCH BY NAME — O(n)

```
SEARCH_BY_NAME(node, name):
  IF node IS NULL THEN RETURN NULL
  IF node.Data.Name == name THEN RETURN node.Data
  result = SEARCH_BY_NAME(node.Left, name)
  IF result IS NOT NULL THEN RETURN result
  RETURN SEARCH_BY_NAME(node.Right, name)
```

DELETE — O(log n) avg, O(n) worst

```
DELETE(node, id):
  IF node IS NULL THEN RETURN NULL
  IF id < node.Data.Id THEN node.Left = DELETE(node.Left, id)
  ELSE IF id > node.Data.Id THEN node.Right = DELETE(node.Right, id)
  ELSE  // node found
    IF node.Left IS NULL AND node.Right IS NULL THEN RETURN NULL
    IF node.Left IS NULL THEN RETURN node.Right
    IF node.Right IS NULL THEN RETURN node.Left
    successor = FIND_MIN(node.Right)   // two children
    node.Data = successor.Data
    node.Right = DELETE(node.Right, successor.Data.Id)
  RETURN node
```

IN-ORDER TRAVERSAL — O(n)

```
IN_ORDER_TRAVERSAL(node):
  IF node IS NULL THEN RETURN
  IN_ORDER_TRAVERSAL(node.Left)
  DISPLAY node.Data
  IN_ORDER_TRAVERSAL(node.Right)
```

LOAD FROM DATABASE — O(n log n)

```
LOAD_FROM_DATABASE(databasePath, tree):
  connection = OPEN_CONNECTION(databasePath)
  query = "SELECT Id, Name, Category, Price, Description FROM MenuItems"
  reader = EXECUTE(query, connection)
  WHILE reader HAS NEXT ROW DO
    item = new MenuItem(reader.Id, reader.Name, reader.Category, reader.Price, reader.Description)
    INSERT(tree.root, item)
  CLOSE(connection)
```

2.5 Database Design

Why SQLite

SQLite was chosen via the *Microsoft.Data.Sqlite* library because it is lightweight, file-based, and requires no server installation (Microsoft, 2024b). The database file path is provided by the user at runtime, satisfying the requirement that no filenames are hardcoded.

Table Structure

Column	Type	Constraint	Description
Id	INTEGER	PRIMARY KEY	Unique identifier; also the BST key
Name	TEXT	NOT NULL	Item name
Category	TEXT	NOT NULL	e.g. Starters, Mains, Desserts, Drinks
Price	REAL	NOT NULL	Item price in pounds
Description	TEXT	—	Optional item description

Normalisation

The design is in First Normal Form (1NF) — each column holds a single atomic value with no repeating groups. A separate categories table could achieve higher normalisation but would add unnecessary complexity for a fixed, small set of values.

Read/Write Operations

Initialise: CREATE TABLE IF NOT EXISTS on startup. Load: SELECT reads all rows into the BST. Save: BST cleared then written back using INSERT OR REPLACE to avoid duplicates. Delete: Parameterised DELETE statements prevent SQL injection.

Persistence

Without the database, BST data is lost on close. SQLite provides permanent storage so additions, updates, and removals are retained across sessions, combining the BST's fast in-memory performance with reliable persistence.

3. Testing

3.1 Testing Approach

The project adopts unit testing using the ***MSTest*** framework (Microsoft, 2024c). Unit testing was selected because it allows individual components to be tested in isolation, ensuring each class and algorithm works independently of the UI or database layer. Tests cover the ***MenuItem*** class (correct instantiation), ***BinarySearchTree*** (insert, delete, search, traversal, count across valid, invalid, edge, and duplicate scenarios), **Order** class (instantiation, item addition, total price), and ***DatabaseHelper*** (initialise, save, load, delete). Tests run independently so a failure in one does not affect others.

3.2 Test Cases Table

Test Case	Input	Expected Output	Actual Output	Result
Insert single item	MenuItem(Id=1, £5.99)	Count = 1	Count = 1	Pass
Insert multiple items	IDs 10, 5, 15	Count = 3	Count = 3	Pass
Insert duplicate ID	Id=1 inserted twice	Count = 1, item updated	Count = 1	Pass

Test Case	Input	Expected Output	Actual Output	Result
Search by ID: exists	Insert Id=10, search 10	Returns "Spring Rolls"	Item returned	Pass
Search by ID: not found	Insert Id=10, search 99	Returns null	Null returned	Pass
Search by name: exists	Search "Spring Rolls"	Returns item, Id=10	Item returned	Pass
Search by name: case insensitive	Search "spring rolls"	Returns item	Item returned	Pass
Delete existing item	Insert Id=10, delete 10	True, Count = 0	True, Count = 0	Pass
Delete non-existing	Insert Id=10, delete 99	False, Count = 1	False, Count = 1	Pass
Delete node with two children	Insert 10,5,15 — delete 10	Count=2, IDs 5&15 found	Both found, Count=2	Pass
GetAllItems sorted	Insert 20, 5, 10	Returns [5, 10, 20]	Correct order	Pass
Empty tree count	No insertions	Count = 0	Count = 0	Pass

4. AI Agent

4.1 AI Agent Design

A rule-based AI agent accessible to customers through a dedicated chat interface. It assists in navigating the menu, finding items, and understanding options — acting as a virtual assistant that reduces manual browsing. The agent interacts directly with the live *BinarySearchTree* instance, so responses always reflect the current menu state. Input is submitted as natural language text and responses are rendered asynchronously on the customer's chat page.

4.2 Implementation

The agent is implemented in *AIAgent.cs* as a standalone class that accepts a *BinarySearchTree* and an *OrderBST* as constructor parameters, keeping it decoupled from the web framework with no dependency on HTTP context, sessions, or controllers.

Decision-Making Approach

On receiving input, the text is converted to lowercase and trimmed, then evaluated against a prioritised sequence of conditions using a *ContainsAny()* helper method. Each condition checks for relevant keywords and executes the corresponding response logic. This approach is transparent, predictable, and requires no external libraries or machine learning models — keeping the implementation fully within the team's own code as required by the brief.

The agent handles: greetings, full menu display, category browsing (Starters, Mains, Desserts, Drinks), item recommendations, cheapest/most expensive item queries, keyword search across item names and descriptions, price queries, order placement direction, order tracking direction, item count, help, and a fallback for unrecognised input.

Integration with the System

The agent is exposed through *AIController*, which receives POST requests from the chat page. The controller instantiates an AIAgent with the shared BST singletons, calls *GetResponse()* with the customer's message, and returns the response as JSON. The view renders it asynchronously without a full page reload.

Limitations

As a rule-based system, the agent can only respond to inputs matching predefined keyword patterns. It has no contextual memory — each input is processed independently with no awareness of previous turns. Complex or ambiguous queries outside its keyword vocabulary return a fallback response. A natural language processing model would be required to address these limitations.

5. Conclusion

5.1 Summary of Work Done

This project successfully delivered a Takeaway Restaurant Management System built as an ASP.NET Core MVC web application. The team implemented two custom Binary Search Trees — one for menu items, one for orders — both designed and built from scratch without standard library implementations. Admin users can manage the full menu and order statuses. Customer users can browse and search the menu, place orders, view order history, and interact with the AI agent. All data is persisted to a SQLite database. A suite of MSTest unit tests was written to verify BST correctness independently of the UI.

5.2 Limitations

The standard BST does not self-balance — insertion in sorted order would degrade performance to O(n). An AVL tree would eliminate this, but the risk is low given non-sequential real-world data. The Order class uses a fixed-size array of 50 items, placing an artificial cap unsuitable for a production system. Additionally, if the application crashes mid-session, any orders not yet written to the database would be lost.

5.3 Critical Reflection

Choosing the BST was well-justified given the requirements for frequent search and ordered display, and its implementation demonstrated genuine understanding of recursive algorithms and tree traversal. Separating business logic from the UI made unit testing straightforward and kept the codebase maintainable.

The main challenge was the app type changing during development — from console to WinForms to ASP.NET Core MVC. While the final result is more capable, this lack of early alignment cost time and introduced inconsistencies. In future, the team would agree on architecture at the outset and document that decision formally. More time would also be invested in designing data structures for all classes before coding begins, and a proper *.gitignore* would be configured from the start.

6. References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*. 4th edn. Cambridge, Massachusetts: The MIT Press.

Microsoft (2024a) *Overview of ASP.NET Core MVC* [Online]. Available at: https://learn.microsoft.com/en-us/aspnet/core/mvc/overview (Accessed: 9 April 2026).

Microsoft (2024b) *Overview — Microsoft.Data.Sqlite* [Online]. Available at: https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/ (Accessed: 9 April 2026).

Microsoft (2024c) *MSTest overview — .NET* [Online]. Available at: https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-mstest-intro (Accessed: 9 April 2026).